

TinyDETR-Pose

Camera-Ready Action Plan

Repository and manuscript audit

9 August 2026

- Executive summary
- 1. Immediate scientific-integrity audit
- 2. Correct positioning relative to prior work
- 3. Repair the evaluation protocol
- 4. Add a second dataset: LM-O
- 5. Ablation program
- 6. Diagnose and explain the ADD(-S) gap
- 7. Validate the efficiency and edge-deployment claims
- 8. Redesign Figures 1 and 2
- 9. Rewrite tables and results narrative
- 10. Expand implementation and reproducibility details
- 11. Manuscript revision map
- 12. Proposed schedule and gates
- 13. Minimum defensible package if time becomes critical
- 14. Final deliverables checklist
- Recommended final message of the paper
- Primary references for the revision

Purpose: Convert the conditional acceptance into a defensible, reproducible camera-ready submission.
Primary outcome: A paper whose method description matches the released implementation, whose efficiency claims cover the complete pose pipeline, and whose accuracy is evaluated with both legacy ADD metrics and official BOP metrics.

Executive summary

The paper has been accepted because the problem is practical and the efficiency-accuracy trade-off is useful. The remaining work is not merely editorial. Before adding experiments, the team must establish exactly which code, checkpoint, matcher, loss, and deployment graph produced every reported number. The current repository contains meaningful differences from the PDF, and the existing BOP and TensorRT paths do not yet reproduce the complete method.

The work should proceed in this order:

1. **Freeze and audit the paper artifact.** Recover the exact commit, checkpoint, configuration, dataset protocol, and evaluation code used for Tables 1–5.
2. **Make code and paper agree.** Decide which loss and matcher are the actual final method, then describe and release that method exactly.
3. **Repair evaluation and deployment.** Produce target-independent BOP predictions and export the complete pose network to ONNX/TensorRT.
4. **Run the mandatory experiments.** Official BOP evaluation, matcher/loss ablations, and a focused analysis of the ADD(-S) gap.
5. **Add LM-O if at all feasible.** It is the most relevant second dataset because it tests occlusion and enables direct comparison with RACE-6D and BOP baselines.
6. **Rewrite positioning, figures, tables, and reproducibility material.** Frame the contribution as an edge-oriented architecture and Pareto point rather than as the first DETR-based direct pose estimator.

Definition of done

The camera-ready is ready only when all of the following are true:

- Every headline number can be regenerated from a tagged commit and a saved configuration.
 - The paper's equations describe the loss that actually trained the released checkpoint.
 - The paper's matching equation describes the matcher that actually trained the released checkpoint.
 - Official BOP scores are generated by the official toolkit from model predictions without ground-truth Hungarian selection.
 - TensorRT latency includes rotation, translation, classification, and any required query filtering/back-projection.
 - Accuracy parity is demonstrated across PyTorch, ONNX, and TensorRT.
 - The main weight choices have empirical sensitivity evidence.
 - The ADD(-S) weakness is quantified and discussed honestly, especially on asymmetric objects.
 - Figures 1 and 2 clearly separate inherited components from proposed/adapted components.
 - The public README permits an external researcher without private NAS access to reproduce training and evaluation.
-

1. Immediate scientific-integrity audit

This is the critical path. Do not start a large ablation sweep until the following questions have written, evidence-backed answers.

1.1 Freeze the submission artifact

Create a **camera-ready** branch and an immutable tag for the exact submission version. Preserve:

- The PDF submitted to the venue.
- The checkpoint used for every main and ablation table.
- The git commit associated with each checkpoint.
- The complete command-line arguments and environment variables.
- Dataset hashes or file manifests.
- Evaluation output directories, raw JSON/CSV files, and logs.
- The TensorRT engine, ONNX graph, export command, and benchmark log used for the latency claim.

Create one machine-readable result manifest per run. Recommended fields:

```
run_id
git_commit
checkpoint_sha256
config_path
seed
dataset_and_split
training_data_sources
validation_split
test_split
metric_implementation
bop_toolkit_commit
hardware
runtime
precision
input_resolution
batch_size
headline_metrics
```

Acceptance criterion: Given a row in any paper table, the team can identify one manifest and one immutable checkpoint that produced it.

1.2 Reconcile the method with the current implementation

The PDF currently describes a uniform ADD-S loss for all objects and no direct component-wise rotation supervision. The current repository instead contains a conventional mixed ADD/ADD-S loss selected using symmetry flags and an explicit 6D rotation loss. The current launcher also enables a full-translation matching cost while the paper describes a projected-center-only pose cue.

Resolve this by answering two separate questions:

1. What exactly produced the submitted results?
2. What method will be claimed and released in the camera-ready?

Do not silently modify the prose to match whichever implementation is easiest. Evaluate the scientifically meaningful alternatives in Section 5, choose one based on validation evidence, rerun the final test evaluation, and make the paper and code agree.

Produce a one-page “paper-to-code mapping” with:

Paper element	Exact implementation	Configuration key	Verification test
Rotation head	Module and output conversion	rotation representation	Orthonormality/determinant test
UV center head	Module and activation	center loss weight	Projection/back-projection round trip
Depth head	Log-depth transform and units	depth loss weight	Known-depth numeric test
Matcher	Exact cost terms and weights	matcher configuration	Synthetic assignment cases
Point-set loss	ADD-S or ADD(-S), mesh sampling	pose loss weight	Symmetric/asymmetric toy meshes
Inference filtering	Confidence and top-k rule	selection parameters	Duplicate/false-positive test

1.3 Verify every reported number and inconsistency

Recompute Tables 1–5 from raw result files rather than copying values manually. Explicitly resolve:

- ADD(-S) AUC reported as both 65.6 and 64.6.
- ADD(-S)@0.1d reported as both 93.5 and 93.6.
- “ADD-S@0.1d” versus “ADD(-S)@0.1d.”
- Parameter count reported as 14.5M versus the count produced by the released model.
- Training length stated as 60 epochs versus the current 100-epoch launcher.
- The claimed 73.5% TensorRT latency reduction versus the 50% reduction from 9 ms to 4.5 ms.
- IoU versus generalized IoU terminology.
- Whether the batch size of 32 is global or per device.

For the unusually low cracker-box ADD(-S) value, audit class IDs, model units, mesh/object origin, symmetry flags, rotation convention, and row alignment in the generated table.

Acceptance criterion: One script generates all final LaTeX/CSV table values directly from immutable result artifacts.

1.4 Eliminate test-set leakage

The PDF states that the 900-image BOP subset of YCB-V test scenes is used for validation while results are reported on the 2,949 YCB-V test keyframes. If checkpoint selection or hyperparameter tuning uses those 900 images, the final test result is not independent.

Adopt a clean protocol:

- Construct validation data only from the training portion, using held-out real training sequences or a fixed train-derived validation split.
- Freeze all hyperparameters and select the checkpoint using this validation split.
- Evaluate the YCB-V test keyframes and BOP target set only once for final reporting.
- Document any unavoidable legacy tuning on test data and rerun the final configuration under the corrected protocol.

Acceptance criterion: No image used for hyperparameter selection or checkpoint selection occurs in the final test evaluation.

2. Correct positioning relative to prior work

2.1 Recommended claim

Use this as the central positioning:

TinyDETR-Pose is an edge-oriented redesign of direct transformer-based 6D pose estimation. It combines a lightweight LW-DETR detector, a shallow pose decoder, image-plane center plus log-depth translation, symmetry-safe assignment, and pose supervision to produce a compact, PnP-free system. Its contribution is the resulting accuracy-latency-parameter trade-off and the analysis of which pose components remain effective under a strict edge budget.

Avoid claiming novelty for set prediction, single-stage direct pose estimation, 6D continuous rotation, projected-center translation, log-depth, or ADD-S individually.

2.2 Required comparison with T6D-Direct

State clearly that T6D-Direct already introduced DETR-style, single-stage, multi-object direct 6D pose estimation and evaluated object-only versus pose-aware Hungarian matching.

Then make the distinction explicit:

Dimension	T6D-Direct	TinyDETR-Pose
Detector foundation	DETR/ResNet-50	LW-DETR/ViT-Tiny
Decoder	Six layers	Three layers
Translation	Direct metric translation	UV center plus log-depth and analytic back-projection
Matching	Class and box selected after ablation	Class, box, and projected center; no ambiguous rotation term in the claimed variant
Pose supervision	Symmetry-aware rotation plus translation	Final formulation to be verified by the loss ablation
Main goal	Real-time direct regression	Edge hardware and compact Pareto point

The projected-center matcher should be described as a limited, image-plane pose cue added to T6D-Direct’s object-only matching—not as an entirely new set-prediction paradigm.

2.3 Required comparison with RACE-6D

Treat RACE-6D as concurrent work if its publication timing overlaps the submission. Its overlap must still be acknowledged.

Dimension	RACE-6D	TinyDETR-Pose
Overall form	Single-pass transformer set prediction	Single-pass lightweight transformer set prediction
Rotation	Keypoint-conditioned coarse and refined rotation	Direct 6D rotation head
Translation	Bounding-box-normalized cues and log-depth refinement	Projected center plus single log-depth prediction
Refinement	Coarse-to-fine residual refinement	No iterative/coarse-to-fine refinement
Inputs	RGB and RGB-D variants	RGB-only
Evaluation	Official BOP and multiple datasets	Must add official BOP; currently YCB-V only
Design goal	Accuracy with fast refinement	Minimum model/latency for edge deployment

The paper should explicitly say that TinyDETR-Pose sacrifices refinement capacity and some pose precision for a smaller and simpler deployment graph.

2.4 Related-work additions

Add a short paragraph covering other prior uses of the same subcomponents, particularly methods that decompose translation into 2D center and depth. Explain that the novelty lies in integration with LW-DETR and the measured edge trade-off.

3. Repair the evaluation protocol

3.1 Keep ADD metrics, but define them precisely

Continue to report:

- AUC of ADD-S.
- AUC of ADD(-S), using ADD for asymmetric and ADD-S/ADI for symmetric objects.
- ADD(-S) accuracy at 0.1 object diameter.
- Per-object values.

For every metric specify:

- Distance integration interval and sampling step.
- Whether the final score is a macro-average over objects or micro-average over instances.
- Exact symmetry list and its source.
- Model-point sampling or full evaluation mesh.
- Units for meshes and translation.
- Matching/selection protocol for predictions.

Do not call conventional ADD(-S) “ADD-S.”

3.2 Add official BOP localization metrics

Use the official BOP toolkit and report:

- AR-VSD.
- AR-MSSD.
- AR-MSPD.

- AR-BOP = mean of the three.

This provides direct comparability to RACE-6D and current BOP literature. Pin and report the exact BOP toolkit commit or release.

3.3 Add official BOP detection metrics if possible

Because TinyDETR-Pose claims to detect an arbitrary set of objects without NMS or known object counts, BOP localization alone is insufficient: localization provides target object identities/counts. Add the current BOP model-based 6D detection evaluation when schedule permits:

- AP-MSSD.
- AP-MSPD.
- Mean BOP detection AP.

At minimum, report standard 2D detection AP and recall to show that pose accuracy is not obtained through oracle detection.

3.4 Build a target-independent prediction exporter

The exporter must not call the training matcher with ground-truth boxes, centers, labels, rotations, or translations. For every query:

1. Convert class logits to calibrated class scores.
2. Select the class and confidence according to a documented rule.
3. Retain predictions using a fixed top-k or confidence rule compatible with the selected BOP task.
4. Export rotation in the correct row-major BOP convention.
5. Export translation in millimetres.
6. Preserve scene ID, image ID, object ID, score, and runtime.
7. Keep the method NMS-free if that is a headline claim.

Write validation tests for:

- A ground-truth prediction file scores approximately 1.0.
- Identity and known 90-degree rotations are exported correctly.
- Metre-to-millimetre conversion is correct.
- Symmetry-equivalent poses receive equivalent BOP errors.
- Low-confidence false positives reduce detection AP.
- Duplicate queries are handled by the official evaluator, not hidden by ground-truth matching.

3.5 Report detection and oracle-matched pose separately

If oracle/ground-truth matching is retained for diagnostic ADD evaluation, label it explicitly as **oracle-matched pose quality**. Do not use it alone to support the end-to-end detection claim.

Recommended table split:

- **End-to-end metrics:** BOP detection/localization, detection AP/recall, no GT matching.
- **Pose diagnostic metrics:** ADD/ADD-S after explicitly stated oracle association, if useful.

4. Add a second dataset: LM-O

LM-O is the preferred second dataset because it tests heavy occlusion, directly probes the stated failure mode, and is supported by the official BOP toolkit.

4.1 Dataset preparation

- Use the eight canonical LM-O objects with their original BOP object IDs.
- Replace hard-coded absolute paths in the conversion script with command-line arguments.
- Preserve original object IDs in BOP export; keep any internal contiguous mapping in a documented mapping file.
- Generate deterministic COCO-style annotations with a recorded converter commit and command.
- Verify camera intrinsics per image rather than relying on one backup matrix.
- Use official evaluation meshes and symmetry definitions from `models_info.json`.
- Record exact real, synthetic, and PBR training sources.

4.2 Training and tuning protocol

- Define a train-derived validation split before training.
- Do not tune on LM-O test images.
- Use the same architecture and weight configuration chosen on YCB-V unless a dataset-specific change is scientifically necessary.
- If a class-specific output layer requires retraining, initialize it using the same documented procedure as YCB-V.
- Keep input resolution, augmentation family, and training schedule as consistent as possible.

4.3 LM-O reporting

Report:

- AR-VSD, AR-MSSD, AR-MSPD, and AR-BOP.
- Per-object BOP scores.
- ADD(-S)@0.1d only as a secondary legacy metric.
- Runtime and model size, which should be architecture-invariant.
- Results stratified by visible fraction if possible.

4.4 Fallback if full LM-O training is impossible

The second dataset was described as ideal rather than absolute in the decision. If compute or time makes full LM-O training impossible:

- Complete official YCB-V BOP evaluation first.
- Add a small, clearly labeled LM-O feasibility experiment only if it follows a valid protocol.
- Do not present zero-shot or partially tuned results as a full benchmark comparison.
- Explain the limitation and release the completed LM-O data/evaluation pipeline for follow-up work.

5. Ablation program

5.1 Rules shared by every ablation

All ablations must:

- Start from the same detector-pretrained checkpoint.
- Use the same data, augmentation, global batch size, optimizer, schedule, and checkpoint-selection rule.
- Use a train-derived validation set, never the test set, for selection.
- Change only the named factor.
- Report parameter count and inference latency if the factor changes architecture.
- Report ADD-S AUC, ADD(-S) AUC, ADD(-S)@0.1d, AR-BOP, and asymmetric-object ADD(-S).
- Use at least three seeds for the final model and the principal baseline. For expensive screening rows, one seed is acceptable if clearly labeled; rerun any near-tie with three seeds.
- Report mean $\pm$ standard deviation for multi-seed rows.

5.2 Ablation A: pose-loss formulation — mandatory

This is the most important ablation because it tests the claimed symmetry contribution and may explain the asymmetric-object gap.

ID	Asymmetric objects	Symmetric objects	Direct rotation loss	Purpose
L0	Direct rotation + translation only	Symmetry-masked/direct baseline	Yes	Lightweight direct-regression baseline
L1	Uniform ADD-S	Uniform ADD-S	No	Exact loss claimed in the submitted PDF
L2	ADD	ADD-S	No	Conventional ADD(-S) pose-level supervision
L3	ADD	ADD-S	Yes, 6D L1 or geodesic	Tests whether direct rotation closes asymmetric gap
L4	ADD	Symmetry-transform-minimized rotation/ADD-S	Yes	Strong symmetry-aware alternative, if implementation is validated

Required analyses:

- Overall and per-object metrics.
- Symmetric versus asymmetric macro averages.
- Mean rotation error on asymmetric objects.
- Mean translation and depth error.
- Gradient magnitude or unweighted loss scale for each component at early, middle, and late training.

Decision rule: The final claimed loss must be selected using the preregistered validation metric. If uniform ADD-S is not best, remove or narrow the claim rather than retaining it for novelty.

5.3 Ablation B: matching cost — mandatory

First isolate the contribution of projected-center matching:

ID	Class	Box L1	GloU	UV center	Depth	Rotation
M0	2	5	2	0	0	0
M1	2	5	2	1	0	0
M2	2	5	2	2.5	0	0
M3	2	5	2	5	0	0
M4	2	5	2	10	0	0
M5	2	5	2	5	5	0
M6	2	5	2	5	0	symmetry-aware rotation cost
M7	2	5	2	5	5	symmetry-aware rotation cost

M0–M4 justify the selected center weight. M5–M7 justify excluding ambiguous depth and rotation from matching rather than merely asserting that exclusion is stable.

Also report assignment diagnostics on validation data:

- Fraction of matches with correct class.
- Mean matched box IoU.
- Mean matched center distance.
- Query-to-instance identity stability across decoder layers or adjacent checkpoints.
- Percentage of assignments changed relative to M0.
- Training divergence or NaN rate.

If full rotation matching is too costly, include at least M0, M3, M5, and one symmetry-aware rotation row.

5.4 Ablation C: loss-weight sensitivity — mandatory

Keep inherited detection weights fixed unless the audit shows otherwise. Perform one-factor sweeps around the final configuration:

Component	Candidate weights
Projected-center loss	1.25, 2.5, 5.0
Log-depth loss	7.5, 15.0, 30.0
Point-set pose loss	10, 20, 30, 50
Direct rotation loss, if retained	0, 0.5, 1, 2

Reuse existing ADD-S-weight runs only after their checkpoint, commit, split, and evaluation provenance have passed the audit.

For each row report both weighted and unweighted mean component losses. This demonstrates whether the numeric coefficient compensates for scale differences or simply dominates optimization.

Recommended compact paper table:

Varied weight	Value	ADD-S AUC	ADD(-S) AUC	Asym. ADD(-S)	AR-BOP
---------------	-------	-----------	-------------	---------------	--------

Move the complete sweep and loss-scale plots to the supplement.

5.5 Ablation D: translation parameterization — strongly recommended

The UV-center plus log-depth decomposition is central to the differentiation from T6D-Direct. Test it directly:

ID	Lateral translation	Depth	Back-projection
T0	Direct metric x,y,z regression	Direct z	No
T1	Normalized UV center	Linear metric z	Yes
T2	Normalized UV center	Log-depth	Yes
T3	Box-relative UV center	Log-depth	Yes, optional RACE-style comparison

Report center error in pixels, relative depth error, translation error in millimetres, convergence curves, and final pose metrics. T0 versus T2 is the essential comparison.

5.6 Ablation E: lightweight architecture choices — strongly recommended

These establish that the efficiency contribution is deliberate rather than inherited entirely from LW-DETR.

Factor	Values	Required outputs
Decoder layers	3, 6	Parameters, FLOPs, latency, AR-BOP, ADD(-S)
Object queries	20, 50, 100	Latency, recall, false positives, AR/AP
Input resolution	Current resolution and one lower resolution	Latency, small-object recall, pose metrics
Backbone scale	ViT-Tiny and one larger LW-DETR scale, optional	Pareto curve

The minimum useful experiment is 3 versus 6 decoder layers. It directly supports the explanation that the shallow decoder causes some accuracy loss while delivering edge efficiency.

5.7 Ablation F: mesh sampling — supplement

Test 128, 512, and 1,024 sampled model points for the point-set training loss. Report:

- GPU memory.
- Training iteration time.
- ADD-S/ADD(-S) and AR-BOP.
- Variance across two mesh-sampling seeds if sampling is not cached deterministically.

This justifies the chosen 512-point setting and improves reproducibility.

5.8 Prioritized run list

If compute is constrained, run in this order:

Tier A — required for the camera-ready

1. Exact submitted configuration reproduction.
2. L1/L2/L3 loss comparison.
3. M0/M3 matcher comparison.
4. Center-cost sweep M1-M4.
5. At least one depth/rotation matching comparison, M5 or M6.

6. Center, depth, and pose weight sensitivity around the selected values.
7. Three seeds for the final method and the strongest directly comparable baseline.

Tier B — strongly recommended

1. Translation T0/T1/T2.
2. Three versus six decoder layers.
3. LM-O final training and BOP evaluation.

Tier C — supplement/future work

1. Query-count sweep.
2. Mesh-point sweep.
3. Larger backbone Pareto point.
4. INT8 quantization or pruning.

6. Diagnose and explain the ADD(-S) gap

The paper should not explain the gap solely through qualitative speculation. Perform the following analyses.

6.1 Symmetric versus asymmetric breakdown

Report four macro averages:

- ADD-S on all objects.
- ADD(-S) on all objects.
- ADD on asymmetric objects only.
- ADD-S on symmetric objects only.

This will reveal whether the strong all-object ADD-S result masks weak orientation on asymmetric objects.

6.2 Error decomposition

For every object and overall, report:

- Rotation error in degrees, using a valid symmetry-aware alternative for symmetric objects.
- Translation error in millimetres.
- Depth error in millimetres and relative percentage.
- Projected-center error in pixels.
- ADD(-S) error distribution, not only AUC.

Use these values to identify whether failure is caused by rotation, depth, image-plane localization, or detection.

6.3 Occlusion and scale stratification

Bin instances by visible fraction, for example:

- 0–25% visible.

- 25–50%.
- 50–75%.
- 75–100%.

Also bin by projected object area or bounding-box size. Report AR-BOP or ADD(-S) for each bin. This tests the paper’s claim that occlusion drives the failures.

6.4 Cracker-box and scissors audit

For these failure-heavy objects:

- Verify object ID and class-name mapping.
- Verify symmetry annotation.
- Verify CAD scale and coordinate frame.
- Check rotation matrix convention and determinant.
- Plot error histograms.
- Inspect the highest-confidence failures and low-confidence successes.
- Compare the loss variants L1–L3.
- Compare three versus six decoder layers if available.

Only after these checks should the paper attribute the gap to occlusion or shallow decoding.

6.5 Rewrite the discussion

State the trade-off quantitatively. A suitable structure is:

1. TinyDETR-Pose nearly matches T6D-Direct on symmetry-tolerant ADD-S.
2. It trails recent baselines substantially on correspondence-sensitive ADD(-S), especially for asymmetric objects.
3. The error decomposition shows which component is responsible.
4. The loss and decoder ablations quantify how much accuracy can be recovered and at what latency/parameter cost.
5. The final method is positioned on this measured Pareto frontier.

7. Validate the efficiency and edge-deployment claims

7.1 Export the full pose graph

The ONNX/TensorRT graph must output at least:

- Class logits or final confidence/class.
- Bounding boxes if used for reporting/detection.
- Rotation representation or final 3×3 rotation.
- UV center.
- Log-depth or metric depth.
- Final translation, either inside the graph or through a measured deterministic post-process.

If back-projection is outside the engine, time and report it separately and include it in end-to-end latency.

7.2 Establish numerical parity

On a fixed validation batch compare PyTorch, ONNX Runtime, and TensorRT:

- Maximum and mean class-logit difference.
- Box difference.
- Rotation geodesic difference.
- UV difference.
- Depth and translation difference.
- Final ADD/ADD-S and BOP metric difference over the validation split.

Predefine tolerances. Investigate any accuracy change larger than normal FP16 variation.

7.3 Use a reproducible benchmark protocol

For each hardware/runtime pair record:

- Exact device model; verify whether it is Jetson Nano, Orin Nano, Xavier, or another platform.
- JetPack, CUDA, cuDNN, TensorRT, PyTorch, and driver versions.
- Power mode, clocks, thermal state, and active cooling.
- Input resolution and batch size.
- FP32, FP16, or INT8 precision.
- Number of warm-up and timed iterations.
- Mean, median, standard deviation, p90, and p99 latency.
- Whether host-to-device transfer, preprocessing, and post-processing are included.
- Peak memory and, if possible, power draw.

Report both **engine-only latency** and **end-to-end application latency**.

7.4 Avoid cross-hardware speed claims

Separate the runtime evidence into two tables:

1. **Published baseline runtimes**, with each paper’s original hardware and protocol clearly listed.
2. **Controlled benchmark**, containing only models rerun under the same hardware, resolution, precision, batch size, and timing scope.

Do not calculate “47% faster” across unmatched hardware. The strongest defensible claim may be “lower reported latency,” not a controlled speedup.

7.5 Recommended runtime table

Model	Device	Runtime	Precision	Resolution	Batch	Engine-only ms	End-to-end ms	Params	AR- BOP
-------	--------	---------	-----------	------------	-------	-------------------	------------------	--------	------------

Add a latency-versus-accuracy Pareto plot with parameter count encoded by marker size.

8. Redesign Figures 1 and 2

8.1 Figure 1: lineage and contribution overview

Replace the generic DETR diagram with a compact comparison:

```
T6D-Direct: DETR + direct R,t heads
              |
              | lightweight detector and pose factorization
              v
TinyDETR-Pose: LW-DETR + R6D + UV + log-z + analytic back-projection
              |
              + projected-center matching
              + selected symmetry-aware pose objective

RACE-6D: set prediction + keypoints + coarse-to-fine refinement
              |
              + explicit contrast: more refinement/accuracy, larger graph
```

The caption should say which ideas are inherited, adapted from prior work, and introduced as part of the final system.

8.2 Figure 2: exact implementation architecture

Use a legend:

- Gray: unchanged LW-DETR.
- Orange: TinyDETR-Pose trainable additions.
- Blue: deterministic geometry.
- Green dashed lines: training-only matching.
- Purple dashed lines: training-only losses.

Show:

- ViT-Tiny encoder and projector.
- Three decoder layers and object queries.
- Class and box heads.
- The exact rotation MLP depth.
- Any shared translation MLP.
- UV and depth output layers.
- Exponential conversion from log-depth.
- Camera-intrinsic back-projection.
- Final R,t pose.
- Exact matcher inputs.
- Exact loss connections.

Do not draw three one-hidden-layer MLPs if the implementation uses a shared translation MLP plus separate projections.

8.3 Optional Figure 3 replacement

Use a Pareto plot rather than another head schematic:

- X-axis: controlled end-to-end latency.
- Y-axis: AR-BOP or ADD(-S) AUC.
- Marker size: parameter count.
- Marker shape: refinement versus no refinement.

This makes the practical contribution visually immediate.

9. Rewrite tables and results narrative

9.1 Main comparison table

Recommended columns:

Method	Input	Single-stage	Refinement	PnP	Params	Device/protocol	ADD-S AUC	ADD(-S) AUC	AR-VSD	AR-MSSD	AR-MSPD	AR-BOP
--------	-------	--------------	------------	-----	--------	-----------------	-----------	-------------	--------	---------	---------	--------

Use “not reported” instead of implying missing results are zero. Clearly separate numbers reproduced by your team from numbers copied from papers.

9.2 Per-object table

- Keep YCB-V per-object ADD metrics in the supplement if space is tight.
- Add symmetric/asymmetric group averages.
- Flag any object whose result is under audit.
- Do not highlight values within 2% as equivalent to best without defining the rule in the caption.
- Generate highlighting automatically from the table data.

9.3 Ablation tables

Use separate tables for:

- Loss formulation.
- Matcher design and center-cost sensitivity.
- Loss-weight sensitivity.
- Translation parameterization and lightweight architecture.

Each table should include AR-BOP and ADD(-S), not only ADD-S.

9.4 Results language

Prefer precise statements such as:

- “Within 0.3 percentage points of T6D-Direct on ADD-S AUC.”
- “10.0 points below T6D-Direct on ADD(-S) AUC.”
- “Reduces decoder depth from six to three and parameter count by X%, at a cost of Y AR-BOP points.”

Avoid “comparable” unless the allowed difference is stated and supported.

10. Expand implementation and reproducibility details

10.1 Paper or supplement checklist

Document:

- Input resizing, padding, normalization, and camera-intrinsic transformation.
- Exact ViT layers, feature indices, projector levels, hidden dimension, attention heads, decoder layers, and sampling points.
- Number of queries at train and test time.
- Group-DETR training configuration and auxiliary losses.
- Exact architecture and initialization of every pose head.
- Rotation conversion and numerical safeguards.
- Depth representation, clamp/epsilon values, and units.
- Back-projection equation and handling of per-image intrinsics.
- Mesh sampling algorithm, seed, point count, and cache behavior.
- Symmetry source and whether symmetry labels/transforms are used.
- Every matching and training-loss coefficient.
- Optimizer, weight decay, all component learning rates, layer decay, gradient clipping, learning-rate drops, EMA, and total epochs.
- Global batch size, GPU count, seed, and number of runs.
- All augmentations and the epoch at which they start.
- Pretraining datasets and exact checkpoint hashes.
- Checkpoint-selection metric and validation split.
- Evaluation and confidence-selection protocol.
- Hardware/software benchmark environment.

10.2 Public repository checklist

Replace the current private-NAS-oriented README with:

- Public clone URL and tested commit/tag.
- Environment creation and exact dependency lock file.
- Docker build instructions that complete without manual edits.
- CUDA operator build and test command.
- Public YCB-V and LM-O download links.
- Dataset conversion commands with expected directory trees.
- Pretrained-weight and final-checkpoint download links with hashes.
- One-command training, validation, ADD evaluation, BOP export, BOP scoring, ONNX export, TensorRT export, and latency benchmark.
- Expected outputs and approximate runtime/storage requirements.
- License/provenance notes for code inherited from LW-DETR and PoET.

Add smoke tests that run on CPU or a small debug dataset.

10.3 Configuration management

Replace manuscript-critical shell-variable combinations with versioned configuration files, for example:

```
configs/camera_ready/ycbv_final.yaml
configs/camera_ready/ycbv_loss_L1.yaml
configs/camera_ready/ycbv_matcher_M0.yaml
configs/camera_ready/lmo_final.yaml
configs/camera_ready/ycbv_eval_bop.yaml
configs/camera_ready/ycbv_export_trt_fp16.yaml
```

At startup, save the fully resolved configuration, git status, package versions, and parameter count in the output directory.

11. Manuscript revision map

Abstract

- Lead with the edge deployment problem.
- Avoid implying that direct transformer set prediction is new.
- Report one official BOP score alongside ADD metrics.
- State device, precision, and whether latency is engine-only or end-to-end.
- Use one consistent value for each metric.

Introduction

- Add T6D-Direct and RACE-6D to the first-page positioning.
- Define the contribution as lightweight redesign and empirical Pareto analysis.
- Remove broad statements that existing single-stage methods lack principled symmetry handling unless carefully qualified.

Related work

- Add a direct comparison paragraph for T6D-Direct.
- Add RACE-6D as prior or concurrent work.
- Discuss prior projected-center/depth decomposition methods.
- Separate methodological novelty from system integration.

Method

- Make equations match the released loss and matcher exactly.
- Define all tensor domains and units.
- Correct the MLP architecture description.
- State which parts are inherited unchanged from LW-DETR.
- Explain why center is included in matching and show the corresponding ablation.

Experiments

- Add a clean data-split statement.

- Add official BOP metrics and toolkit version.
- Add LM-O protocol/results.
- Add the required ablations.
- Split controlled runtime results from published runtimes.
- Add statistical variation and seed information.

Limitations

- Explicitly acknowledge the ADD(-S)/asymmetric-object gap.
- Explain the edge-accuracy trade-off using measured ablations.
- Mention class-specific training and dependence on CAD models/symmetry metadata if those remain true.
- Do not defer requested evaluation entirely to future work.

Conclusion

- Emphasize demonstrated edge efficiency and simple deployment.
- Avoid “best” claims unless supported under a common protocol.
- Use the corrected parameter, accuracy, and latency values.

12. Proposed schedule and gates

This schedule assumes access to the existing multi-GPU training environment. Adjust calendar duration to actual cluster availability, but preserve the dependency order.

Week 1 — provenance, protocol, and infrastructure

Days 1-2

- Freeze submission artifacts and create run manifests.
- Recover exact configurations for every reported table.
- Establish the clean train/validation/test split.
- Decide the canonical metric definitions.

Days 3-4

- Repair target-independent BOP export.
- Add synthetic unit tests and run official BOP scoring on ground-truth/perturbed fixtures.
- Repair full-pose ONNX/TensorRT export and parity tests.

Day 5 gate

Proceed only if:

- A submitted checkpoint reproduces its claimed ADD metrics within a predefined tolerance.
- BOP export passes unit tests.
- Full-pose TensorRT output matches PyTorch numerically.

Week 2 — mandatory ablations

- Run loss formulation L1/L2/L3.
- Run matcher M0/M3 and the center-cost sweep.
- Start the depth/rotation matcher comparison.
- Run one-factor loss-weight sensitivity.
- Analyze symmetric/asymmetric results as runs finish.

Week 2 gate

- Select the final loss and matcher using validation results.
- Lock all final hyperparameters.
- Launch three final seeds.

Week 3 — final evaluation and LM-O

- Evaluate final YCB-V seeds with ADD and official BOP metrics.
- Train/evaluate LM-O using the locked configuration.
- Run translation parameterization and three-versus-six decoder comparisons if resources permit.
- Complete controlled runtime measurement and edge benchmark.

Week 4 — paper and release

- Regenerate all tables from result artifacts.
- Redraw Figures 1 and 2 and produce the Pareto plot.
- Rewrite positioning, method, experiment, and limitation sections.
- Complete public README, configuration files, checkpoint links, and environment lock.
- Conduct an internal paper-to-code audit by someone who did not implement the method.

Final submission gate

Two team members independently verify:

- Every table value against raw outputs.
- Every equation against code.
- Every runtime claim against benchmark logs.
- Every comparison against the cited source and protocol.
- Every public command on a clean environment or container.

13. Minimum defensible package if time becomes critical

If schedule pressure forces cuts, do not cut the provenance audit, code-paper reconciliation, or official BOP evaluation. The minimum acceptable package is:

1. Exact reproducible paper checkpoint and configuration.
2. Correct, target-independent official YCB-V BOP metrics.
3. Uniform ADD-S versus ADD(-S) versus ADD(-S)+rotation ablation.

4. Center matcher off/on plus a center-weight sensitivity sweep.
5. Honest asymmetric-object and ADD(-S) gap analysis.
6. Revised T6D-Direct/RACE-6D positioning and contribution figures.
7. Full-pose TensorRT verification or a corrected/narrowed latency claim.
8. Corrected tables and a public reproduction README.

LM-O, the full architecture sweep, and quantization can be reduced only after these items are complete.

14. Final deliverables checklist

Code and artifacts

- ☐ Camera-ready git tag.
- ☐ Exact final YCB-V and LM-O configuration files.
- ☐ Final checkpoints and SHA-256 hashes.
- ☐ Per-run result manifests.
- ☐ Target-independent BOP exporter.
- ☐ Official BOP raw result CSVs and score JSONs.
- ☐ Full-pose ONNX and TensorRT engines.
- ☐ PyTorch/ONNX/TensorRT parity report.
- ☐ Controlled latency logs.
- ☐ Unit and smoke tests.

Experiments

- ☐ Final YCB-V ADD metrics.
- ☐ Final YCB-V official BOP metrics.
- ☐ LM-O official BOP metrics, if feasible.
- ☐ Loss-formulation ablation.
- ☐ Matcher design and center-weight ablation.
- ☐ Loss-weight sensitivity.

- ☐ Translation parameterization ablation.
- ☐ Three-versus-six decoder ablation.
- ☐ Symmetric/asymmetric and occlusion analysis.
- ☐ Three-seed final result.

Paper

- ☐ Revised title/abstract claims if necessary.
- ☐ Explicit T6D-Direct and RACE-6D positioning.
- ☐ Revised Figures 1 and 2.
- ☐ Official BOP comparison table.
- ☐ Separate controlled and published runtime tables.
- ☐ Corrected per-object and ablation tables.
- ☐ Expanded implementation details.
- ☐ Revised limitations and ADD(-S) discussion.
- ☐ All numeric/text inconsistencies fixed.

Release

- ☐ Public installation and dataset instructions.
- ☐ Public pretrained weights and checkpoints.
- ☐ One-command reproduction paths.
- ☐ Dependency lock or reproducible container.
- ☐ License and attribution audit.

Recommended final message of the paper

The strongest defensible story is:

Direct transformer pose estimation does not need a large detector, deep refinement stack, PnP, or NMS to be practically useful. By adapting LW-DETR with a compact pose parameterization and carefully validated assignment and supervision, TinyDETR-Pose provides a reproducible edge-oriented Pareto point. Official BOP and ADD evaluations show both its strengths and its remaining loss of fine asymmetric-object accuracy, while controlled ablations quantify the sources of that trade-off.

That story is practical, honest, and well aligned with why the reviewers accepted the work.

Primary references for the revision

- TinyDETR-Pose submission PDF: [TinyDETR-Pose.pdf](#) in this repository.
- T6D-Direct: <https://arxiv.org/abs/2109.10948>
- RACE-6D: https://openaccess.thecvf.com/content/CVPR2026F/html/Ha_RACE-6D_Real-time_Accurate_Coarse-to-fine_Object_6D_Pose_Transformer_CVPRF_2026_paper.html
- BOP task and metric definitions: <https://bop.felk.cvut.cz/tasks/>
- Official BOP toolkit: https://github.com/thodan/bop_toolkit